

PAYTMFLOW ARCHITECTURAL KNOWLEDGE PACKAGE

PaytmFlow — Developer & Operations Playbook

Document Type: Day-to-Day Engineering Manual & Operational Runbook

Status: Authoritative Repository Audit

Target Audience: Frontend Developers, Backend Engineers, Full-Stack Contributors, QA Engineers

Primary Rule: Strict Directory Ownership (`frontend/` vs `backend/`) & Frozen API Contract

1. Prerequisites & Tooling Setup

Required System Software

- **Node.js:** `v20.x` or `v22.x` (LTS)
- **Python:** `3.12.x` or `3.14.x` (Managed via `uv`)
- **Package Manager (Python):** `uv` (`curl -LsSf https://astral.sh/uv/install.ps1 | iex` on Windows)
- **Docker & Docker Compose:** Required for local PostgreSQL 16
- **Git:** Standard git client

Initial Clone & Directory Layout

```
git clone https://github.com/PrasadBant/PaytmFlow.git
cd PaytmFlow
```

2. Quick Start: Local Development

Option A: Independent Frontend Development (Mock Mode - Default)

Dev1 can build and test the entire frontend UI without running PostgreSQL or FastAPI:

```
cd frontend
npm install
npm run dev
```

- Open <http://localhost:5173>
- Intercepted by **MSW 2.0** service worker responding from `contract/fixtures/`.
- No backend process required.

Option B: Full-Stack Integrated Local Development (Live API Mode)

1. Start Backend & Database

```
# In Terminal 1
cd backend
docker compose up -d postgres
uv sync
uv run alembic upgrade head
# On Windows PowerShell:
uv run uvicorn app.main:app --port 8000 --loop none
# On Linux/macOS:
make api
```

2. Start Frontend in Live Mode

```
# In Terminal 2
cd frontend
# Set VITE_API_MODE=live in frontend/.env or inline:
npm run dev
```

- Vite dev server automatically proxies `/api` requests to `http://localhost:8000`.

3. Testing, Linting & Typechecking Commands

Backend Verification Suite

```
cd backend

# 1. Run all 290 pytest tests
uv run pytest

# 2. Strict type check on Deterministic Core (Invariant Gate)
uv run mypy --strict app/core

# 3. Formatting & Linting
uv run ruff check app tests

# 4. Architectural Boundary Enforcement (import-linter)
uv run lint-imports
```

Frontend Verification Suite

```
cd frontend

# 1. Run Vitest unit & component test suite (314 tests)
npm test -- --run

# 2. Strict TypeScript type check (tsc --noEmit)
npm run typecheck

# 3. ESLint & Accessibility check (jsx-a11y)
npm run lint

# 4. Production bundle build verification
npm run build

# 5. Playwright E2E Golden Path walk
npx playwright test
```

4. Debugging & Troubleshooting Runbook

A. Debugging Session & Cookie Issues

- **Symptoms:** API returns `404 NOT_FOUND` on existing journeys or new session minted unexpectedly.
- **Root Cause:** Session cookies (`pf_session`) are signed with `SESSION_SECRET` . If the secret changes or cookie is cleared, the backend safely mints a new anonymous session.
- **Remedy:**
 1. Verify browser cookie `pf_session` exists under Application → Storage → Cookies.
 2. In automated tests/cURL, pass `X-Session-Id: 00000000-0000-0000-0000-000000000001` (valid when `APP_ENV=local` or `ci`).
 3. Ensure `fetch()` calls in frontend include `{ credentials: 'include' }` (handled centrally in `frontend/src/api/client.ts`).

B. Debugging `409 ACTION_STALE`

- **Symptoms:** Submitting an action returns `409 Conflict` with `ACTION_STALE` .
- **Root Cause:** The client submitted `expected_snapshot_id` from Version `$N$`, but the server has already transitioned to Version `$N+1$`.
- **Remedy:**
 1. Inspect `error.details.current_snapshot_id` returned in the response.
 2. Confirm frontend displays the amber banner: *"This journey has moved on — refreshing"*.
 3. Verify TanStack Query invalidates `['journey', id]` and refetches the latest snapshot.
 4. **Do NOT blindly retry** the action with the new snapshot ID without user re-confirmation.

C. Debugging Evidence Upload & PyMuPDF Extraction

- **Symptoms:** Document upload fails or extracts blank text.
- **Root Cause:** Corrupt PDF stream or unsupported MIME type.
- **Remedy:**
 1. Check uploaded file size (must be `$\le 10\text{MB}$`). Files over 10MB fast-fail with HTTP `413` .

2. Test text extraction locally using Python CLI:

```
uv run python -c "import pymupdf; doc=pymupdf.open('path/to/test.pdf'); print(doc[0].get_text())"
```

- For image uploads (JPEG/PNG), the system generates a simulated binary verification record per `backend/app/evidence/extract.py`.

D. Debugging Mock Mode Scenarios (MSW)

- The frontend supports URL-based scenario overrides in mock mode:
- `http://localhost:5173/j/11111111-1111-1111-1111-111111111111?scenario=stale` → Simulates `409 ACTION_STALE`
- `http://localhost:5173/j/11111111-1111-1111-1111-111111111111?scenario=needsreview` → Simulates `INCOME_MISMATCH` ambiguity
- `http://localhost:5173/j/11111111-1111-1111-1111-111111111111?scenario=deadend` → Simulates unrecoverable `DEAD_END`
- `http://localhost:5173/j/11111111-1111-1111-1111-111111111111?scenario=aitimeout` → Simulates AI gateway timeout

5. How to Extend PaytmFlow

A. Adding a New Journey Pack

- Author the Manifest:** Create `backend/app/packs/manifests/<pack_name>.yaml` defining `metadata`, `goal_schema`, `state_schema`, `dependencies`, `actions`, `evidence_mappings`, and `ambiguity_rules`.
- Validate Manifest Integrity:**

```
uv run pytest backend/tests/packs/test_all_manifests.py
```

- Generate Fixtures:** Add corresponding mock responses in `contract/fixtures/<pack_name>/`.
- Zero Frontend Code Needed:** The generic frontend automatically renders the new pack in Screen 2 and compiles dynamic forms for Screen 3.

B. Updating the API Contract

- Edit Contract:** Modify `contract/openapi.yaml`.
- Validate Schema Invariants:**

```
uv run pytest backend/tests/contract/test_openapi_matches.py
```

- Regenerate Frontend TypeScript Types:**

```
cd frontend
npx openapi-typescript ../contract/openapi.yaml -o src/api/types.gen.ts
```

- Update Fixtures:** Synchronize any updated fields in `contract/fixtures/`.

C. Adding a New UI Primitive

- Create `frontend/src/components/primitives/<ComponentName>.tsx`.

2. Export from `frontend/src/components/primitives/index.ts`.
3. Add interactive demo in `frontend/src/screens/KitchenSink.tsx` (`/dev/kitchen-sink`).
4. Write unit test in `frontend/src/components/primitives/primitives.test.tsx` verifying keyboard focus and aria roles.

6. Pre-Commit & Pre-Demo Checklists

Pre-Commit Checklist

- [] Backend: `uv run pytest` passes (290 tests green).
- [] Backend: `uv run mypy --strict app/core` is clean (0 errors).
- [] Backend: `uv run lint-imports` keeps all architecture boundaries.
- [] Frontend: `npm run typecheck` passes (0 errors).
- [] Frontend: `npm run lint` passes (0 warnings).
- [] Frontend: `npm test -- --run` passes (314 unit tests).
- [] Integrity: No changes made to generated `types.gen.ts` by hand.
- [] Claim Safety: Automated claim scan tests pass (no forbidden marketing words).

Pre-Demo Checklist (2 Minutes to Showtime)

1. Reset Database State:

```
curl -X POST http://localhost:8000/api/v1/demo/reset -H "X-Demo-Secret: change-me-demo-secret"
```

2. **Confirm AI Mode:** Ensure `AI_PROVIDER=mock` in `backend/.env` for deterministic, zero-latency demo presentation.
3. **Disable Dev Badges:** Ensure `VITE_SHOW_DEV_BADGES=false` in `frontend/.env`.
4. **Verify Flagship Flow:** Walk Screens 1 → 9 on `LENDING` to verify happy path completion.

7. Related Documents

- [Complete Project Guide & Master Document](#)
- [Architecture Reference Manual](#)
- [Shared Development Contract](#)
- [Canonical OpenAPI Specification](#)